

Consistency Parsing vs Dependency Parsing - (Phrase Structure)

Feature	Consistency	Dependency
1) Definition	Divides a sentence into hierarchical phrases	Identifies grammatical relationships between individual words
2) Focus	Phrase structure	Word to word dependencies
3) Rep.	Parse tree with phrases	Dependency tree with words
4) Root	Entire sentence (S)	Main verb (ROOT)
5) Nodes	Words & phrase labels	Only words are nodes
6) Complexity	More complex	Simpler
7) eg	NP → "The boy" VP → "eats an apple"	eats → boy (subject) eats → apple (object)
8) App.	Grammar checking, machine translation, Syntax analysis	Information extraction, Chatbots Semantic analysis

Morphology -

- Studies the internal structure of words & how words are formed from smaller meaningful units called morphemes
- Reduces vocabulary size
- Improve search, translation & text analysis

Types of Morphology -

(a) Inflectional : changes the grammatical form of a word without changing its meaning

eg - play → playing

(b) Derivational : creates a new word by changing meaning or grammatical category

eg - happy → happiness

Transducers -

- Type of Finite State Machine (FSM) that reads an input string & produces an output string

- Transforms one sequence into another

- 3 steps :

└ Reads input symbol

└ Changes state

└ Produces the corresponding output symbol

eg - playing → play

(i) Start : Transducer receives the input word "playing"

(ii) Check suffix : Checks whether the word ends with a known suffix

(eg : -ing, -ly)

(iii) Remove suffix: identifies -ing & removes it
(iv) Output: remaining root (stem) is "play"

Language Model -

- Computational model that learns the patterns, structure & relationships between words in a language.
- Goal: determine which word is most likely to come next
- Uses probability with chain rule for prediction

eg: Input: The sun rises in the

The model assigns probabilities to possible next words:

east $\rightarrow 0.95$	} Prediction: east
morning $\rightarrow 0.03$	
sky $\rightarrow 0.01$	
ocean $\rightarrow 0.01$	

- Types:

└ Statistical Language Model

└ Neural Network Based Language Model

└ Transformer Based Language Model

N-Gram -

- Contiguous sequence of 'N' items like words or characters from text or speech
- Value of 'N' determines the order of N-gram
 - Unigrams (1-grams) are single words
 - Bigrams (2-grams) are pairs of consecutive words
 - Trigrams (3-grams) are triplets of consecutive words

~~##~~ ~~6~~

Corpus: "In consciousness studies the consciousness studies are fundamental."

Training an N-Gram: Unigram Model -

Set of unigrams: { "in", "consciousness", ..
..... "fundamental" }

Probability of unigram, $P(w) = \frac{C(w)}{m}$

where, $C(w)$ = count of words

m = total size of corpus (8)

eg. $P(\text{"consciousness"}) = \frac{\text{count}(\text{"consciousness"})}{m}$
 $= 2/8$

Training an N-Gram: Bigram Model -

Set of bigram: { "in consciousness",
..... "are fundamental" }

Uses conditional probability

$$P(y/x) = \frac{C(x, y)}{\sum_w C(x, w)} = \frac{C(xy)}{C(x)}$$

$$\text{eg. } P(\text{"are"} | \text{"studies"}) = \frac{\text{count}(\text{"studies are"})}{\text{count}(\text{"studies"})} \\ = 1/2$$

Training an N-Gram: Trigram Model -

Set of trigram: { "in consciousness studies",
..... "studies are fundamental" }

Uses conditional probability of the third word

$$P(w_3 | w_1^2) = \frac{C(w_1^2 w_3)}{C(w_1^2)} \text{ where, } C(w_1^2 w_3) = C(w_1 w_2 w_3) = C(w_1^3)$$

$$\text{eg. } P(\text{"the"} | \text{"conscious studies"}) \\ = \frac{\text{count}(\text{"consciousness studies the"})}{\text{count}(\text{"consciousness studies"})} \\ = \frac{1}{2}$$

Limitation of N-Gram -

- Limited context
- Data sparsity
- Zero probability problem (occurs when a lang. model assigns a probability of 0 to a word or sequence because it never appeared in the training data)
- Large memory requirement
- Cannot handle unknown words

Generalization -

- Ability of an Ngram language model to predict or assign probabilities to new sentences or word sequences that were not seen during training

Solution: Smoothing -

- To overcome zero probability problem, smoothing techniques assign a small non zero probability to unseen word sequences

- Types:

- (i) Laplace (Add-One) Smoothing
- (ii) Good Turing discounting
- (iii) Absolute discounting
- (iv) Interpolation

Techniques:

1) Laplace (Add One) Smoothing -

- Add 1 to every count

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i) + 1}{\text{count}(w_{i-1}) + V}$$

count of bigram
V = vocab size total

V = vocabulary size → count of history word

2) Add K Smoothing -

- Add a small constant K instead of 1

3) Good Turing Smoothing -

- Redistributes probability from frequent events to unseen ~~data~~ events

RNN -

- Recurrent Neural Network

- Processes data one word/token at a time & passes information from previous steps to future steps

- Problem: As sequence become longer, RNN forgets earlier info (vanishing gradient) problem

LSTM -

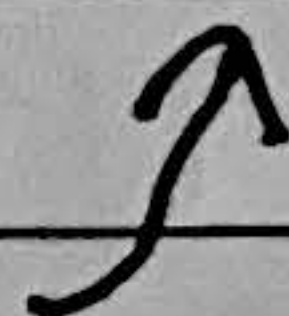
- Long Short Term Memory
- Special type of RNN that can learn long term dependencies

Need for Transformers (BERT, GPT) -

- Overcome limitations of RNNs & LSTMs

- Sequential processing
- Difficulty remembering very long context
- Fixed length memory
- Limited parallelization

Solved by
attention
mechanism



- Allows every word to directly look at every other word

Attention assigns different ~~mechanisms~~ importance (weights) to different words

Evaluating Language Model -

- Measures how well it predicts or generates natural language

(a) Intrinsic evaluation: Measures quality of the language model itself

(b) Extrinsic evaluation: Measures how useful the language model is in a real application

Intrinsic evaluation -

1) Sentence probability:

- Probability assigned to an entire sentence
- Indicates how likely the model believes the sentence is
- Uses chain rule of probability

2) Log probability:

- To avoid numerical underflow & simplify calculations, use log of probability

$$\log P(S) = \sum \log(P(w_i | \text{history}))$$

Higher (less negative) $\rightarrow$ Better model

Lower (more negative) $\rightarrow$ Poor model

3) Cross entropy:

- Measures average uncertainty while predicting next word

$$H = -\frac{1}{N} \sum \log_2 P(w_i) \quad \left| \begin{array}{l} N = \text{no. of words} \\ P(w_i) = \text{probability assigned to the correct word} \end{array} \right.$$

Low value $\rightarrow$ Better predictions

High value $\rightarrow$ More uncertainty

4) Perplexity:

Measures how confused the model is while predicting the next word

$$PP = 2^H$$

$$PP = P(w)^{-1/N}$$

H = Cross Entropy.
 N = No. of words

↓
Lower value $\rightarrow$ Better model
Higher value $\rightarrow$ Poor model

Extrinsic evaluation -

- 1) Machine translation $\rightarrow$ BLEU score
- 2) Text summarization $\rightarrow$ ROUGE score
- 3) Chatbots $\rightarrow$ Human evaluation
- 4) Sentiment analysis } Accuracy, Precision,
- 5) Text classification } Recall, F1 score

#English Word Class -

- Group of words categorized according to their grammatical function, meaning & role in a sentence

- 2 categories:

(a) Open Class Words

- Can easily accept new ~~pos~~ words

- Mainly carry lexical meaning

- Classes: Nouns, ~~Vets~~ Verbs, Adjectives, Adverbs

(b) Closed Class Words

- Contain relatively fixed set of words

- New words rarely added

- Mainly perform grammatical or structural functions

- Classes: Pronouns, determiners, prepositions, Conjunctions

#BIO Tagging Scheme -

A common method for named entity tagging is the BIO scheme

B = Beginning: marks first word of an entity

I = Inside: marks a word that is inside an entity

O = Outside: means word is ~~part~~ not part of any named entity

eg. Ankita works at Chandigarh University

Ankita → B-PER

works → O

at → O

Chandigarh → B-ORG

University → I-ORG

B = Beginning

PER = Person

O = Outside

~~B = Beginning~~

ORG = Organization

I = Inside

Rule based tagging -

- Assigns a POS tag to a word by looking at the word itself & its surrounding context

- Steps:

(i) Use a dictionary to find possible POS tags of a word

(ii) Apply handwritten rules to decide which tag is appropriate

- Limitations / disadvantages:

- High cost

- High time complexity

- Not scalable

Stochastic based tagging -

- Uses probability & statistical information learned from a labeled corpus to assign the most likely POS tag to each word

- Techniques:

(a) Word frequency measurements:

- For ambiguous word, choose the POS tag that has occurred most frequently with that word in the training corpus

(b) Tag & sequence probability:

- Tagger predicts the POS tag of an ambiguous word by looking at the POS tags of the words that come before it
- Considers how likely a particular tag is to occur after a given sequence of previous tags

← (c) Hidden Markov Model (HMM) for POS tagging:

- Determines the most probable sequence of POS tags for a given sentence
- POS tags are hidden
- Markov assumption: Next state depends only on the current state, not on entire history
- Uses
 - Transition probability
 - Emission probability

(a) Transition Probability = how likely is one POS tag to follow another POS tag

(b) Emission Probability = how likely is a particular word given a particular POS tag

- Optimizing HMM with Viterbi algorithm:

- Efficiently finds the most probable sequence of hidden POS tags for a given sequence of words

- Only keeps the best score for each tag at each position

- Uses dynamic programming

- Limitations/disadvantages of Stochastic based approach:

- Requires labeled corpus (training data)

- No probability for words that don't exist

- Uses different testing corpus other than training corpus

Transformation based tagging -

- Combination of rule based & stochastic tagging methodologies
- Algorithm keeps on searching for the new best set of rules given input as labeled corpus

- Input — [Tagged corpus
 [Dictionary

- Output: Sequence of transformation rules

- Brill Tagging algorithm:

└ Error driven transformation based POS tagging that starts with initial tags & learns contextual rules to correct tagging errors iteratively

└ Steps:

└ First assigns initial tags

└ Identifies errors

└ Learns a transformation rule

└ Applies the rule

└ Repeats until no useful rule remains

└ Drawbacks / Limitations:

└ ~~Time~~ Training / learning time complexity is high

└ Does not provide tag probabilities